

LECTURE-12

Monday
5/10/26

 Bucket Sort: Even distribution of data

- 1) Create $\uparrow$ buckets evenly (much smaller size of each than entire data)
- 2) Store data in buckets and sort (use *insertion sort*)
- 3) *Copy* data from bucket into original array.

*We assume each bucket has evenly distributed data (almost same no. of data in each).

→ Array of linked List.

Date _____

data: 91, 12, 23, 37, 33, 05, 90

0 → 05

1 → 12

2 → 23

3 → 37 → 33

...

9 → 91 → 90

Use insertion sort here on linked list using iterators and then it will be:

3 → 33 → 37

9 → 90 → 91

* After this we'll just copy the values from start of linked List into Original.

Code: Bucket Sort ($A[1:n], n$) $\rightarrow$ $B[1:n-1]$ array ?

① for $i \leftarrow 0$ to $n-1$

// make empty buckets $O(n)$

② for $i \leftarrow 1$ to n

// copy values into Array of linked List $O(n)$

③ for $i \leftarrow 0$ to n

// sort each buck with 'k' elements each using Insertion sort

④ Copy data back in original $O(n)$

Time Complexity Analysis:

$$T(n) = O(n) + \sum_{k=0}^{n-1} O(k^2)$$

$$T(n) = O(n) + 2 - \frac{1}{n}$$

$$T(n) = O(n) + \frac{2n-1}{n} = 2n-1$$

$$T(n) = \boxed{O(n)} \text{ linear}$$

$P = 1/n$ (any value will go in one out of n buckets)

$$q = 1 - 1/n$$

$$E[K] = \text{var}[K] + E^2[K] \quad ①$$

$$\text{var}[K] = npq \quad ②, \quad E[K] = np \quad ③$$

$$E[K^2] = n \left(\frac{1}{n} \right) \left(1 - \frac{1}{n} \right) + \left[n \cdot \frac{1}{n} \right]^2$$

$$E[K^2] = 1 - \frac{1}{n} + 1 = \boxed{2 - \frac{1}{n}}$$

Dynamic Programming : For optimising overlapping ↑

Fibonacci Series: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

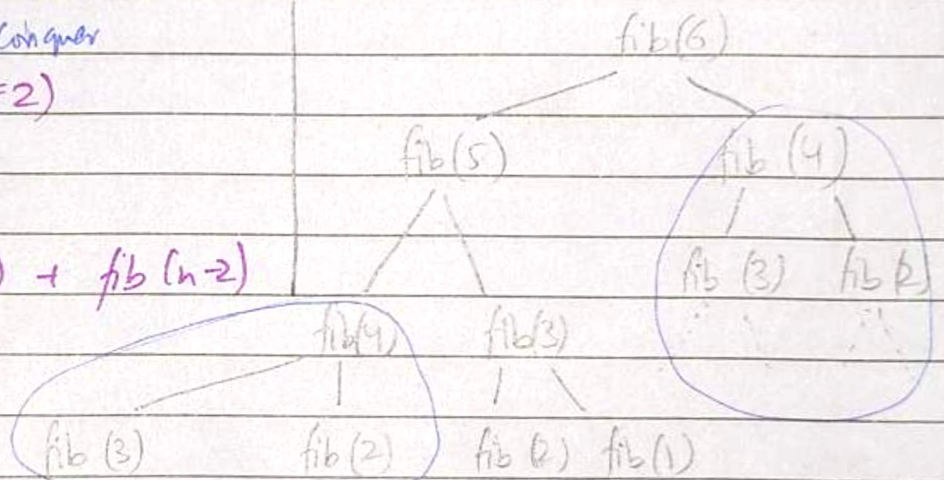
fib(n) { // Divide & Conquer

if (n==1) or (n==2)

return (n-1)

else

return fib(n-1) + fib(n-2)



Problem: The tree shows fib(4), fib(3) and many others are redundant and exactly overlapping. $T(n) = 2^n$ *very Bad*

→ Top down Recursive solution (using DP)

Solution: for such loser $T(n)$; we'll store the result upto a specific 'n' in a stored memory.

M

-1	-1	-1	-1	...
1	2	3	4	...

fib(n, M[n]) {

if m[n] != -1 {

return m[n]

if (n==1) or (n==2) {

~~m[n]~~ m[n] = n-1

return n-1

} else

m[n] = fib(n-1) + fib(n-2)

return m[n]

}

* First time; it goes till fib(6) & (3) and it returns 0 and 1.

Then it'll store 0 and 1 in the 1st & 2nd index of M:

M

0	1	-1	-1	...
---	---	----	----	-----

* Then call goes for fib(2)

that requires fib(1) & fib(0)

but this time it'll just take

values from 'M' array and

provide to fib(2) and write again in

M

0	1	1	-1	...
---	---	---	----	-----

Pearl Notes